This article was downloaded by: [171.67.34.205]
On: 24 June 2013, At: 23:29
Publisher: Taylor & Francis
Informa Ltd Registered in England and Wales Registered Number: 1072954 Registered office: Mortimer House, 37-41 Mortimer Street, London W1T 3JH, UK



IIE Transactions
Publication details, including instructions for authors and subscription information:
http://www.tandfonline.com/loi/uiie20

INTEGRATED SHOP FLOOR CONTROL USING AUTONOMOUS AGENTS

GRACE YUH-JIUN LIN [a] & JAMES J. SOLBERG [a]
[a] School of Industrial Engineering, Purdue University, West Lafayette, IN, 47907
Published online: 30 May 2007.

To cite this article: GRACE YUH-JIUN LIN & JAMES J. SOLBERG (1992): INTEGRATED SHOP FLOOR CONTROL USING AUTONOMOUS AGENTS, IIE Transactions, 24:3, 57-71

To link to this article: http://dx.doi.org/10.1080/07408179208964224

PLEASE SCROLL DOWN FOR ARTICLE

Full terms and conditions of use: http://www.tandfonline.com/page/terms-and-conditions

This article may be used for research, teaching, and private study purposes. Any substantial or systematic reproduction, redistribution, reselling, loan, sub-licensing, systematic supply, or distribution in any form to anyone is expressly forbidden.

The publisher does not give any warranty express or implied or make any representation that the contents will be complete or accurate or up to date. The accuracy of any instructions, formulae, and drug doses should be independently verified with primary sources. The publisher shall not be liable for any loss, actions, claims, proceedings, demand, or costs or damages whatsoever or howsoever caused arising directly or indirectly in connection with or arising out of the use of this material.

INTEGRATED SHOP FLOOR CONTROL USING AUTONOMOUS AGENTS

GRACE YUH-JIUN LIN

JAMES J. SOLBERG

School of Industrial Engineering, Purdue University, West Lafayette, IN 47907

In this paper, we present a generic framework for controlling the work flow in computer controlled manufacturing systems. Based on a market-like model and a combination of objective and price mechanism, the framework allows heterogeneous job objectives, admits job priorities, recognizes multiple resources types, and allows multiple step negotiation between parts and resources. The framework is designed to accommodate frequent changes in the environment such as machine failures, tool shortages, and process requirement variations. An object-oriented simulation system is built to demonstrate the flexibility and effectiveness of the proposed framework. The results show that the proposed framework provides a foundation for highly adaptive, real time shop floor control.

Downloaded by [171.67.34.205] at 23:29 24 June 2013

■ This paper is about integrating shop flow control in a highly uncertain and changing manufacturing environment. We envision a manufacturing control and scheduling system that is different from the traditional approach. Instead of machines processing jobs according to a plan established by a global controller, a population of "intelligent" entities operates cooperatively to achieve individual goals. And thus a higher degree of distribution, robustness, modularity, maintainability, simplicity, and efficiency results.

Traditional control and scheduling approaches — the optimization paradigm, the data processing paradigm, and the control paradigm under subsystem optimization or hierarchical decomposition — have supported a great deal of successful work. However, they do not fully account for important aspects such as the uncertainty and complexity of the real world manufacturing environment [38]. Recognizing the distributed nature of the environment and supported by advanced computer technology, researchers have started to explore alternative control and scheduling architectures. Cooperative systems [41], heterarchical structures [13], object-oriented programming concepts [5], [25], [31], real time negotiation of resource assignments, and opportunistic scheduling concepts [12] have been proposed.

The cooperative system approach recognizes the fact that very complex systems are beyond direct control. Instead, they operate through the cooperative behavior of many interacting subsystems which may have their own independent interests, values, and modes of operation. The resulting behavior of the entire system is collectively determined. A heterarchical structure provides flexible and efficient interactions among all entities, while chaotic behavior can be avoided by defining some well-formed, simple rules. Object-oriented programming is the environment in which all things are represented as (software) objects and the actions are carried out by messages sent or received between objects. Internally, an object may carry out its function in ways which are "hidden" from other objects in the systems. These properties ensure that objects will be portable, modular, and maintainable. Real-time negotiation and opportunistic scheduling, as opposed to preplanning, blend well with the above concepts.

A successful model that is capable of handling the complexity and uncertainty of the real world manufacturing systems, however, requires the integration of all the above approaches. Also, massive communication flow may become the bottleneck of the system, and myopic decision making because of loss of global information may occur. The success of the implementation relies on the organizational principles that will ensure harmonious operation and effective completion of work. In this article, we propose a highly adaptive integrated flow control framework which uses the above concepts and is based on a price and objective mechanism. System equilibrium, flexible control strategies, and resources (including machine tools, cutting tools, pallet, fixtures, and transporters) integration scheme are addressed. The resulting system is not only more practical for real applications, but also more flexible and effective. System performance is evaluated through an object-oriented simulation system.

In the following section, we will examine the manufacturing flow control problem. We will review the related cooperative and negotiation-based work in both computer task allocations and manufacturing control and scheduling. Our framework will then be presented and

This paper handled by Michael Shaw.

 0740-817X/92/$3.00x.00 © 1992 "IIE"

discussion of the simulation results and conclusion will be given in the last two sections, respectively.

Manufacturing Floor Control

A modern factory floor often consists of various machine tools linked by various material handling systems from different makers with various degrees of automation and flexibility (Figure 1). Computers or microcomputers are embedded in equipment to control the operation of the equipment, which can also be used as a data storage, communication channel, or decision-making tool when equipped with appropriate software. This environment naturally forms an interconnected and interdependent system where both information and decision making are often logically and geographically distributed.

Floor control is responsible for the coordination of physical flow and information flow in the manufacturing shop floor environment. It interfaces with upper-level factory planning and management modules, low level device control modules, and various other design and control modules such as CAD/CAM, layout design, warehouse control and inventory control to carry out part and resources assignment. Figure 2 illustrates the relationships of floor control with other manufacturing components. It must be able to adapt to the changes and failures of these components, respond to external factors such as order variations, vendor changes, and product design changes, and accommodate multiple changing criteria involving cost, quality, time, and system flexibility.

Criteria for Manufacturing Floor Control

A flow control system under such a versatile and flexible environment needs to satisfy the following criteria:

Distributed Control. Because of the complexity and uncertainty of the manufacturing environment, neither central decision nor global data storage is feasible. Control must be distributed throughout the system so that local decisions can be made to avoid hotspots and utilize local information. Also, the use of distributed computing provides the opportunity for simultaneous decision making and speeds up the response time.

Robustness. The factory must be able to continue operation despite unexpected events.

Modularity. The system must be able to accept new features gracefully and be able to function so that replacement of portions of the equipment has minimal side effects.

Stability of the Concept. The framework remains constant even when parts of the system change.

Efficiency. The system needs to utilize resources well to satisfy customer needs.

Integrated Control. An integrated framework is needed to utilize system resources effectively. Subsystem optimization such as part-machine scheduling, isolated tool management, and isolated transporter control can cause system under-performance. Availability of different resources should be considered in decision making and the most critical resource should have the greatest impact in decision making.

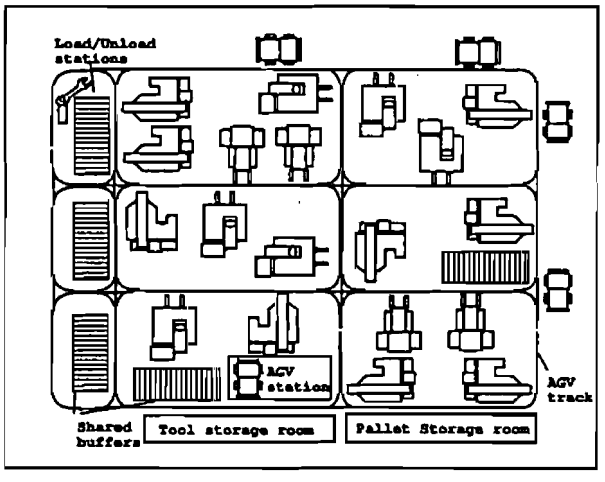


Figure 1. Typical automated manufacturing system

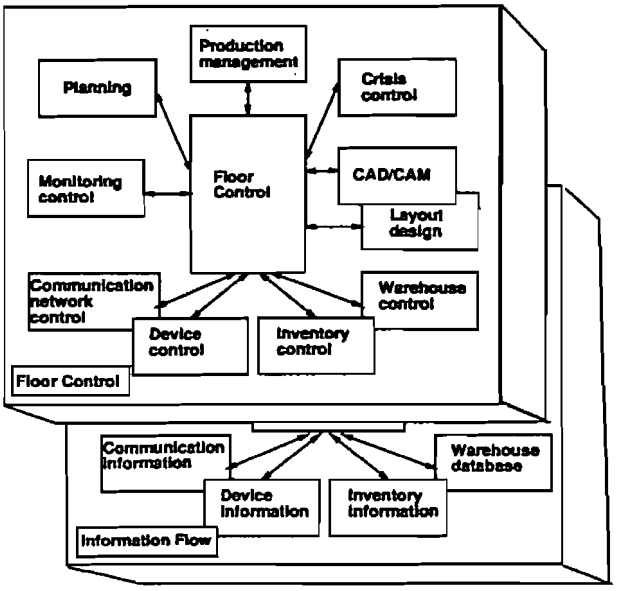


Figure 2. Manufacturing floor control

Our Flow Control Approach

Based on the above criteria, we developed an integrated and flexible control framework to achieve real

Downloaded by [171.67.34.205] at 23:29 24 June 2013

time coordination in a constantly changing environment utilizing the system's intelligence and flexibility to meet customers' needs.

An object-based architecture is used to keep the framework separate from system details so that changes can be made without affecting the functionality of other components. Negotiation offers a reasonable way to use local knowledge in achieving reasonable performance and maintaining system robustness, modularity, and stability. An opportunistic approach allows coordination of intelligent entities through negotiation to achieve individual goals. System flexibility and effectiveness is also explored in the following areas: an input part process representation scheme that allows alternative routing; highly adaptive control methodologies that incorporate changes in the objectives and environment easily; distributed decision making that explores concurrency and robustness; and distributed information flow that minimizes the information traffic.

In the next section, we will review related research. Details of the integrated framework will be presented in a later section.

Related Research

Research in negotiation-based cooperative systems is concerned with how autonomous agents can interact effectively. In 1972, Farber and Larson proposed the use of a bidding scheme for computing resource allocation in a distributed computing system [7], [8]. Smith and Davis refined the idea into the contract net [35], [36] and introduced a general negotiation protocol for communication and control among cooperating agents, with one agent announcing the availability of tasks and awarding them to other bidding agents. Although much of the original work with the contract net is defined in the context of a distributed sensor system [4], the contract net has been extensively studied as a means for distributing tasks among distributed computer processors and controlling and scheduling of manufacturing systems.

Computer Task Allocations

Malone *et al.* [22] developed a prototype system called Enterprise based on contract nets for sharing tasks among personal workstations connected by a local area network. Stankovic and Sidhu [37] developed bidding algorithms assigning larger processes to sites that have more free memory, matching longer processes (those that require more CPU time) to hosts that are relatively less busy, and combinations of the two. Ramaritham and Stankovic [30] developed a bidding algorithm for scheduling computer tasks with hard real-time constraint tasks that must meet their deadlines on computer processors in a network. Using the concept of Microeconomics, Ferguson *et al.* [9] designed an economy for load balancing in a distributed system. In the economy, competition sets prices for the resources in the system; jobs compete for the resources by issuing bids; and the resource allocation decisions are made through auctions held by the processors. Waldspurger *et al.* [42] designed an open, market-like computational system, called Spawn, based on a price mechanism which runs in a distributed network of a heterogeneous high-performance system. It is reported that this market-like resource allocation method based on a price mechanism works effectively in a real distributed system.

Cooperative Manufacturing Control and Scheduling

The early work in distributed flow coordination was the data flow model presented by Lewis [15], and Lewis, Barash, and Solberg [16]. Under the data flow model, machines select jobs according to the first-in-first-out dispatching rule in the internal queue; jobs are routed to the first machine available to complete the next task to be performed on them. No supervisory controller is needed.

Shaw [33], [34] proposed a network-wide bidding scheme for the coordination of the execution of tasks among flexible manufacturing cells. Parunak *et al.* [28], [29] developed a prototype factory control system, called Yet Another Manufacturing System (YAMS), that apportions tasks by negotiation based on the factory hierarchy used in the Automated Manufacturing Research Facility of the National Bureau of Standards [1], [23]. Duffie and Piper [6] studied a heterarchically controlled machining cell by programming parts and processing stations as intelligent entities. Maley and Solberg [20], [21] developed a distributed, object-oriented manufacturing scheduling/control system called CADENCE—the Computer Automated Distributed Environment for Network Coordination and Execution. Ow *et al.* [26] designed a cooperative scheduling system among a group of cooperative problem-solving agents based on contract negotiation. Upton [40] and Ting [39] also suggested using a bidding scheme for the control and scheduling of manufacturing systems.

An Integrated Flow Control Framework

In this section, we will describe an adaptive control and scheduling system framework. The framework is based on distributed information processing, distributed decision making, and a heterarchical market-like model.

Downloaded by [171.67.34.205] at 23:29 24 June 2013

That is, each functional unit of parts and resources is equipped with an intelligent (software) agent who acts as the representative of the entity. The agents communicate and negotiate with other agents in real time through message passing and a bidding protocol to achieve mutual agreements for task sharing (Figure 3). The agents can be considered to be information filters which acquire and recognize factors pertaining to decision making and make decisions based on these current factors. No information aggregation and no advanced planning is required. The intelligence of the agents comes from the specific information about its clients, task specific algorithms, knowledge in its knowledge base, and the experience accumulated in the system (learning).

The success of the implementation of the market-like model lies in the organizational principles and highly adaptive algorithms that ensure the harmonious operation and effective completion of work. These are accomplished in this research by the use of a generic bid construction model based on objectives and a price mechanism, an efficient distributed resources unification scheme, the design of heterogeneous system control components, and highly adaptive algorithms.

The system assumptions will be discussed first followed by a description of the model. The system architecture will then be presented and communication requirements to support the proposed architecture will be discussed. We will discuss some system features associated with the architecture and end with a summary.

Model Assumptions

The integrated flow control framework is designed to address the highly uncertain and changing computer controlled manufacturing environment. Of primary concern is the coordination of information to carry out the entity flow through a facility in a correct and timely manner. More specifically, a job comes to the system with process requirements: process plan, priority, and objective. The controlling and scheduling process will arrange the needed resources including machines, tools, pallets, fixtures, and transporters to get a job done according to the processing requirement to satisfy the part objective, coordinate the resource sharing for jobs in the system, and the information flow within the coordinating process and the communication with other system components.

Each functional unit of the facility or job is considered as a system entity. A functional unit can be a part, a container of parts that is moved as a unit and processed as a batch, a workstation, a transporter or a transporter system controlled by a single controller, a tool magazine controller, fixture or pallet controller. Each entity is attached or connected to a control unit. A control unit is realized in the system by a control computer which communicates with other control units through a communication network to control the entity flow. Any computer language, operating system, and communication protocol can also be used to implement the control framework as long as the message interface is understood by each transmitter and receiver.

A flexible processing process plan [3], [17] is used to describe the process requirement. Using the flexible processing process plan, all possible alternative operations and sequencing of operations to complete a part can be considered in real time to make routing decisions. The use of a flexible processing process not only im-

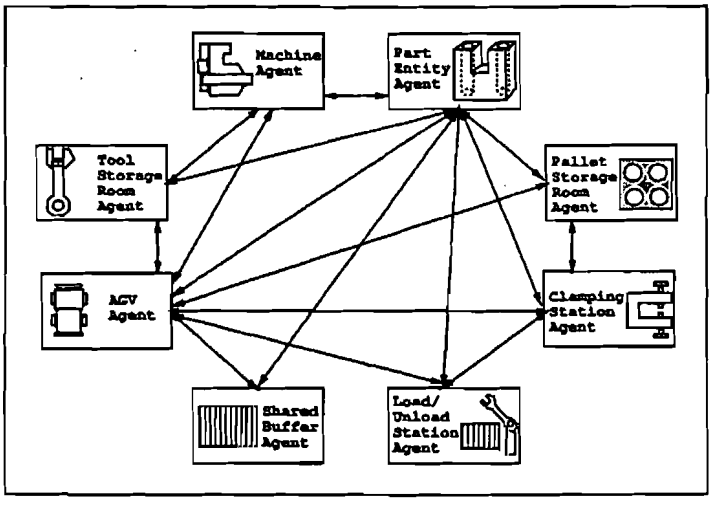


Figure 3. Communication among system agents

Downloaded by [171.67.34.205] at 23:29 24 June 2013

proves production productivity [18] but can also improve the system adaptivity. For example, if a tool is not available for an operation, an alternative tool can be used; if all machines and tools for performing a particular operation are busy, then an alternative operation can be performed as long as it does not violate the precedence constraints.

System Modeling

In this section, we will present our system modeling including the market-like model, a generic bid construction mechanism based on a combination of price and objective mechanism, a multiple-way and multiple-step negotiation metaphor, and a part centered, critical resources based, dynamic resource unification scheme.

Market-Like Model

The model we propose here closely resembles an open marketplace: a job enters the system with some (fictitious) currency and a flexible process plan. It tries to fulfill the processing requirement to achieve its unique set of weighted objectives (time, cost, quality, etc.) by "bargaining" with resource agents. A resource agent acting as a vendor sets its processing charge according to its status and tries to sell its services to maximize its profit and/or some system performance criteria.

The flexible process plan is generated by the CAD/CAM software module. The parts objective reflects a customer's need. Each part, or even each operation of a part, is allowed to have its own objective. The objective can be the minimization of the flow time, the cost of producing the part (or batch of parts), or the maximization of the part quality, or the weighted combination of the above factors. Part currency can be a function of parts priority, direct cost (plus operational cost so far), tardiness cost, or finished goods price.

A Generic Bid Construction Mechanism—A Combination of Price and Objective Mechanism

Under our market-like modeling, parts objectives are transformed into a set of evaluation functions. The weights of the functions can be adjusted dynamically on the basis of the system state and external conditions. The resource agents can adjust their charge prices by their capability and the current global and local system states. Mutual selection and mutual agreement are made through two-way communication.

Research in scheduling up to date either seeks to optimize a fixed system objective such as mean flow time, throughout, tardiness, or uses a price system to guide the search procedure [24], and hence, neglects either the heterogeneous job objectives or job priority. The underlying assumption is that the system has a single criterion to meet. Our proposed combined price and objective mechanism responds to heterogeneous job objectives, job priority, and other factors important for decision making and provides a dynamic environment for the agents to cooperate. It allows parts to devote their currency flexibly to the most important and appropriate resources while resources can adjust the charge prices dynamically based on their status to optimize their utilization. It also models the real world constraints into the limited currency that a part agent can spend. It is generic in the sense that all system factors can be transformed into the weighted evaluation functions and prices, and control/scheduling algorithms and methodologies can be modeled easily by this mechanism.

The incorporation of a price system is also a distinct feature of our system compared to other negotiation-based manufacturing control and scheduling systems. Based on microeconomics theories, the price serves as an invisible hand to guide the negotiation to improve system performance. Under well-defined agent utilities and rules, the control/scheduling system is expected to function based on the equilibrium of the objectives and prices. For example, if the system contains a powerful machine which can complete some operations much faster than other machines, then it will attract many part agents to send their parts to this machine. Very soon, the queue will build up and the expected waiting time will increase. Accordingly, the prices that the machine charges will be increased and the objective values of the interested parts will be decreased. These factors will drive parts to other machines and diminish the demand for this machine. The result is that the system will adjust itself smoothly based on the flow and load of the system, and the overall performance of the system results from the equilibrium of the price system and orchestration of all components involved.

Different price systems and rules will result in different control strategies and system performance. Prices and objectives can be set by an expert, an evaluation scheme, or an expert system and are independent components of the framework. This provides an environment for exercising self-learning.

Multiple-Way and Multiple-Step Negotiation

All decisions are made through multiple-way and multiple-step negotiation. It is a multiple way negotiation since a part usually needs the simultaneous service of several resources such as machine, tool, fixture, and "just-in-time" service of the transporter. Therefore, a part agent will negotiate with different types of resources at a time. It is also a multiple step negotiation

Downloaded by [171.67.34.205] at 23:29 24 June 2013

since both parts and resources have opportunities to deny or accept an offer during the negotiation process based on individual concerns. Final agreement is made only when parts and resources all commit themselves through multiple-step negotiation.

Figure 4 depicts the multiple step negotiation between a part and a resource. That is, the part agent activates task announcement when triggered events arrive. These events include part arrival, near completion of an operation, resource breakdown after commitment, etc. Those resource agents who receive the announcement construct bids based on their status and system states and submit bids if they so desire. Parts may receive many bids. They will evaluate the bids and prepare an offer to the chosen resources. When a resource gets an offer, it has the opportunity to accept or deny based on certain circumstances. The negotiation is completed when both part and resource are committed.

Dynamic Resource Unification Scheme

An important issue during the negotiation process is ensuring that all needed resources for a task are available at the right place and at the right time. This is done by using a part centered, critical resource based, dynamic resource unification scheme. That is, both part and resources have the ability to activate the bidding procedure, and the critical one is the one to drive the bidding procedure. However, since a part has the detailed processing requirement information, the final resources matching decision is done by the part. More specifically, knowing the available resources and their charge price through task announcement and bid collection, a part agent will find a best matching for all required resources to maximize its weighted objective function and minimize the resource charge price. It will make arrangements for transporter, machine, tool, fixture, etc., to be available for the operation at the right place and right time. On the other hand, the resource will select parts to serve to maximize its profit; the decision of the most critical resource will play the most important role in the part resources matching. For example, 1) if a required tool is a bottleneck resource, i.e., with a high charge price and a long waiting time, then, if an alternative tool exists, the alternative tool will be chosen for the service of the operation even if the precision is lower. However, if that is the only tool which can perform the operation, the part agent will have to take the time slot and pay the price that the tool

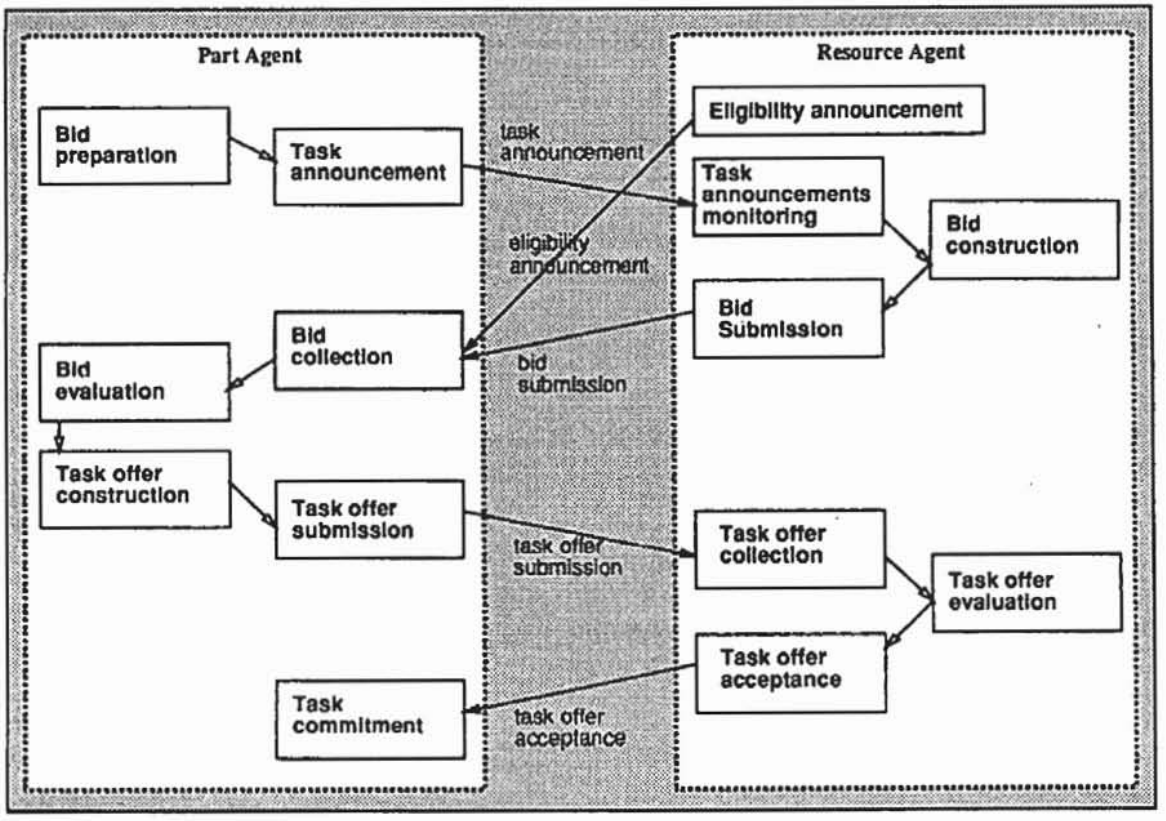


Figure 4. Part-resource negotiation

Downloaded by [171.67.34.205] at 23:29 24 June 2013

agent sets, and try to reserve other resources for that period. The tool agent, realizing the criticality by its high processing charge and long queue, will try to sequence the tasks to maximize profit using the algorithms in the intelligence file. 2) If an AGV controller is incorporated into the system and it is a critical resource, then the part agent will try to arrange other required resources according to the AGV schedule given by the AGV controller.

Under this resource unification scheme, both parts and resources are aggressively participating in the negotiation procedure to achieve their individual goals through mutual selection; the most critical resource will have the biggest impact on decision making. Since a part only needs one machine and one tool, etc., at a time, this decision-making scheme can decrease information flow, simplify a decision-making procedure, and avoid some deadlocks. Note that this approach is very different from the subcontractor approach. In the subcontractor approach, a hierarchical scheme is used. For instance, a part first searches a machine. Therefore, premature commitment due to the lack of second or third level resources, or conflicts due to several machines searching for the same second or third level resources for the same part, often occurs. The original system conditions that led to the current decisions may have been changed by the time a subcontract is obtained. Thus, renegotiating is needed, and this causes system inefficiency.

The use of a price system and the resource unification scheme also resolves bottleneck problems. Bottleneck resources have been recognized as an important factor in system performance, and they play the central role in several scheduling systems such as OPT [10], [11] and OPIS [25]. The central theme is that the throughput of bottleneck resource determines the throughput of the system. However, a bottleneck resource does not necessarily span the entire scheduling horizon and tends to shift before the system is entirely scheduled. Therefore, Adams *et al.* [2] propose a shifting bottleneck procedure for job shop scheduling; Sadeh and Fox [32] propose a micro-opportunistic job shop scheduling approach with a shorter planning horizon to better utilize the shifting bottleneck resource. Under our modeling, the bottleneck resources which can be a machine, tool, transporter, etc., are continuously monitored throughout the system. The system will not only fully utilize the bottlenecks, but will also drive jobs to other machines (if other options exist) to utilize system flexibility and enhance system performance.

The Contract Net Protocol designed by Smith [35] allows the system to be driven by task announcements when the net is lightly loaded and by node availability announcements when the net is heavily loaded. This is because a task-initiated scheduling procedure performs better in a lightly loaded system, and a node-initiated scheduling procedure performs better in a busy system. However, in a distributed environment, global states such as the up-to-date system load and critical resources are not available to the local processors. This makes it difficult, if not impossible, to apply the above approach. It is also worth pointing out that the overall system load does not necessarily reflect the load of the critical resources. The use of our price mechanism and the cooperating part and critical resources decision making process addresses this issue in a rather sophisticated manner. The price fluctuates according to the system state and thus reacts rapidly to the change of system load. In a lightly loaded system, a part may get many low price bids. It can choose resources that fit it best. In a heavily loaded system, critical resources will get many requests and, therefore, get chances to increase the service charge and explicitly (through part selection) and implicitly (through price adjustment) select which part to serve.

To summarize, the proposed model not only has the adaptivity offered by market-like, negotiation-based modeling, but it is also designed to allow heterogeneous job objectives, admit job priorities, recognize multiple resource types, and allow multiple step negotiation between parts and resources to ensure system effectiveness.

System Architecture

The control modules in the control architecture include part agents, resource agents, intelligence agents, monitor agents, communication agents, and database management agents. In the following sub-sections, we describe the organization and responsibilities of each module in detail.

Part Agents

When a part (or a functional unit of parts) enters the system, a part agent is created with a list of processing requirements, currency, and objectives. It manages the part throughout the period when the part is in the system. The agent is an active process that resides in a control computer. The control computer can be a very small one which is attached to the part or pallet and is only responsible for the particular part, or it can be any fixed or non-fixed computer in the system that runs the part process. In the latter case, a new agent process is created in the new control computer, but it inherits all the knowledge of the current agent. In all cases, the

Downloaded by [171.67.34.205] at 23:29 24 June 2013

part agent of a part identifies the part by its part ID which is unique for each part in the system.

The part agent has a local database called an *agent data file*, a local knowledge base called *agent intelligence file*, and an inference engine (Figure 5). The agent data file records information of the part such as part identity number, part process plan or a link to the dynamic process planning system, current part position, part history, part objective (cost, time, due day, quality requirement, etc.), weights of the objectives, and the amount of currency that the part possesses. It also records the current "state" of the system. The part intelligence file stores the current state of the bid in progress and built-in algorithms for the part agent to make decisions during the negotiation process. More complicated or intelligent algorithms and algorithms with different characteristics are stored in a distributed knowledge base that is manipulated by part intelligence agents which will be described in more detail in a later section. The inference engine has the ability to invoke a negotiation process and can perform reasoning processes and enhance the agent's capability by acquiring alternative algorithms from the distributed knowledge base through a simple bidding protocol.

The negotiation process for parts to select resources includes task announcement, bid collection, bid evaluation, task offer submission, and task commitment. The task of each procedure is described as follows.

Task Announcement. Task Announcement solicits bids from related entities. The decisions involved in this procedure include timing constraints, the amount of information to be released in the package, and the selec-

Downloaded by [171.67.34.205] at 23:29 24 June 2013

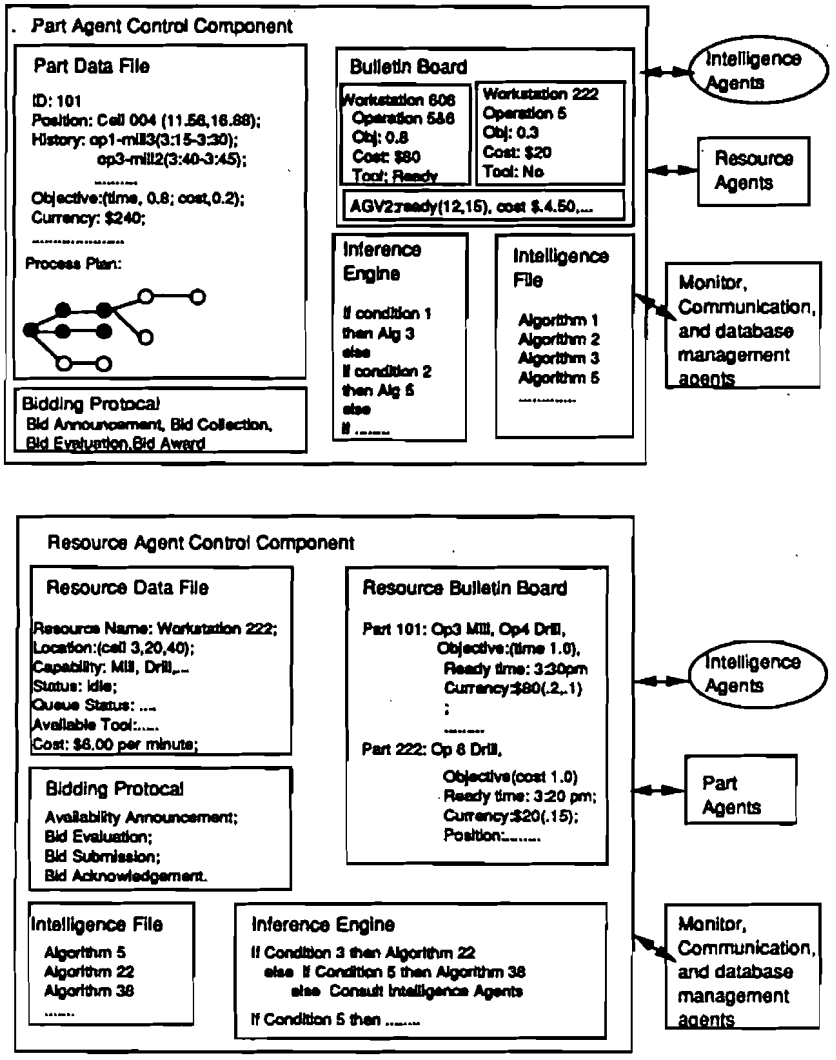


Figure 5. Part agent and resource agent control components

tion of audience. The timing constraint decisions include when to initialize bidding, when to reannounce the task if no qualified bids are received, and how long before the end of a current operation to announce tasks for the next operation. The decisions about information released include what the next possible operations are and how many steps ahead to announce. This problem becomes more interesting and complicated when network process plans or dynamic process plans are used. Audience selection involves selecting resources to solicit bids, i.e., to determine the targets of the task announcement. These can be all types of resources required, such as machines, tools, and transporters, all resources for a particular type of resource, or only a selected subset of these resources (this is called the audience restriction). The decision should be made according to the network configuration, the communication power, prior experience, or rules in the knowledge base.

Bid Collection. After the bid announcement, the part agent collects bids for a certain period of time. It will also collect resource availability information from the network. If any of the agents of the selected resources did not reply, it either assumes that the resource is no longer available or sends an acknowledgment request to the agents asking for reply.

Bid Evaluation. After all bids are collected, the part agent evaluates all available resources using the inference engine and algorithms in its file. If several possible next operations and several look-ahead steps are announced, the process becomes much more complicated. A metric to determine the merits of different bids for different decomposition of tasks is discussed in [19].

Task Offer Submission. After bid evaluation, the part agent selects an operation and offers the operation to the resource agent who submitted the most attractive bid and can optionally inform other resource agents who are not chosen.

Task Commitment. After the task is awarded and task offer acceptances are received from the agents of the resources, the task is committed. The part agent will then arrange for part movement and update the part status and the part history file. If offer acceptance is not received during a period of time, the agent can either retransmit the award message or choose the next desired resource, or restart the bidding procedure, depending on the system state and control algorithms.

All built-in algorithms for the above processes may be adjusted according to the system states by an expert or by rules in the knowledge base. Alternatively, a learning module can be attached to select the appropriate algorithms. Part agents can obtain the specialized algorithms by matching the pre-conditions and post-conditions of the algorithms using a simple bidding procedure similar to the bidding of resources: the part agent announces the requirements of the algorithm using the bid announcement protocol; the part intelligence agents with matching algorithms then submit bids from which the part agent chooses one. The part agent then incorporates the newly acquired knowledge into its inference engine. Under this model, the bidding procedures are black-boxes where different algorithms can be inserted or removed. This allows the characteristics of the procedures to be changed dynamically.

Resource Agents

A "resource" is either a single resource such as a machine, a tool, a fixture, a transporter, or a collection of resources that is viewed as a functional unit or controlled by a single controller such as an AGV controller or tool magazine controller. Each resource is represented by an agent who resides in a control module. Similar to the part agent, each resource agent has associated with it an agent data file, agent knowledge file, agent's inference engines, and the ability to participate in the negotiation process to sell the processing time (Figure 5). The resource agent file records resource information such as price, position, equipment status, queue status, remaining tool life, machine capability, reliability, precision, and records of all pending bids that have been submitted. The resource intelligence file stores built-in algorithms for the resource agent to use during the negotiation process to establish a reasonable selling price. More detailed algorithms with different characteristics are stored in a distributed knowledge base and manipulated by resource intelligence agents. Similar to the parts inference engine, the resource inference engine controls the invocation of negotiation procedure and the selection of algorithms to suit the changing environment.

The bidding procedures for resource management include availability announcement, task announcement monitoring, bid construction, bid submission, task offer collection, task offer evaluation, and task offer acceptance.

Availability Announcement. The resource agent may announce the availability (or expected available time) when the resource becomes idle (or a certain period before the expected available time). In the latter case, the advance announcement timing is controlled by an algorithm in a resource knowledge base. In addition to the availability, it also includes information about the service charge.

Task Announcement Monitoring. This module will collect task announcement messages and pass them to

Downloaded by [171.67.34.205] at 23:29 24 June 2013

the bid construction module every prescribed time interval.

Bid Construction and Bid Submission. After receiving task announcement, the bid construction module retrieves needed information from the resource file. It checks the requirements of the operation and its resource status, evaluates the part price and weighted objective to determine whether to submit a bid using either a built-in algorithm or algorithms in a resource intelligence agents' file. If several parts request the same resource, the resource may adjust its price to maximize its profit. The conflicts in multiple requests can also be solved by using a one-machine sequencing algorithm. Finally, if the resource decides to submit a bid, it calculates and submits needed information including the charged price, expected service time, accuracy, number and types of operations that can be done in a sequence, queue status, reliability, etc., to the part agent. The bid information will be kept in a file for a period of time for later reference. If the resource agent decides not to submit a bid, it will either reply with an invitation refusal message with an optional explanation or take no action, depending on the control policies. The bid information may be used to adjust a threshold for later bid submission. The bid information will be deleted if the award message does not arrive for a certain period (file aging) or a "bid rejection" message is received.

Task Offer Collection and Evaluation. After the resource agent submits bids to parts, it will wait for a period of time to collect task offers. The task offer evaluation then evaluates the offers and chooses an offer by using the inference engine and algorithms in the intelligence file.

Task Offer Acceptance. After task offer evaluation, the agent sends task offer acceptance and/or rejection messages to the corresponding part agents. It then starts preparing physical movement if needed. If any unexpected event which changes the conditions occurs, it will try to renegotiate with the part agent. They can either make a minor adjustment or void the bid award. In the latter case, the part agent can either use previous information to search for substitutions or restart the whole bidding procedure. After the bid acknowledgment is sent, the agent updates the resource status and monitors the execution of the job until the operation is completed.

Intelligence Agents

Intelligence agents are responsible for the manipulation of knowledge bases. The intelligence knowledge base contains a collection of algorithms used by parts or resource agents to make decisions. These decisions include when to announce a bid, how to construct bids, next operation(s) selection, bid evaluation, bid award, etc. For each function, there may be several associated algorithms with different characteristics. For example, it can be either a fast brute-force approach without look ahead (to be used when quick response is required or when the computing power is limited) or a more sophisticated algorithm with global considerations (to be used when the computing power is sufficient). Some may even have the capability to learn from past experience. There are pre-conditions, post-conditions, advantages, and shortcomings associated with each algorithm. After the part or resource agent sends out an algorithm request bid which specifies the objectives, the intelligence agent will match the objectives with the precondition and post condition of the algorithms in the knowledge base and choose one or more to send to the part agent. In certain implementations, the intelligence agent may actually run the algorithms for the part. In this case, the part sends the parameters to the intelligence agent and the latter sends back the results (decisions). A link can be created between the part agent and the knowledge base for later communication if necessary. Note that the algorithm selection procedure or the built-in algorithm overwritten procedure are controlled by part or resource built-in inference engines. And the intelligence of agents can be enhanced by an input of new algorithms, a learning module built in the part or resource intelligence file, and experience accumulated from monitor agents.

The intelligence agents can be distributed in control computers which are independent from the part and resource agents. They communicate with part or resource agents through simple bidding protocols and provide them the needed knowledge or algorithms. The distributed approach makes the system robust; even if some of the computers are down, the remaining system can still function smoothly. Changes can be made and algorithms can be added or removed from the system easily without having to modify other parts of the system or disturb the operations of the manufacturing systems.

Monitor Agents

In a truly distributed environment, all entities in the environment make decisions independently on the basis of the local information and the very limited global knowledge they possess. Although accurate global information may improve the effectiveness and efficiency of the decision-making process, the centralized nature of the global information is exactly what makes the distributed decision making attractive and thus should be avoided. Our solution to this conflict is to let the en-

Downloaded by [171.67.34.205] at 23:29 24 June 2013

tities make decisions independently but provide certain timely global information to the entities upon request. This task is accomplished by distributing a few monitor agents in the system. The monitor agents watch the information flow around their assigned regions and collect and analyze the information to provide timely and accurate global information. They are also responsible for other global-oriented tasks such as setting up job priority, price and currency, maintaining consistency between part currency and resource charge price, and collecting statistics, scheduling maintenance, searching non-active jobs and resources, and interfacing with other manufacturing components and agents within the control component. It can provide part, resource, and intelligence agent knowledge accumulated from history and change the agent's default control algorithms.

Communication Agents

A communication agent resides in each control computer. The duties of the communication agent include: distribution of incoming messages, sending of outgoing messages, and arranging for arrival and departure of the agents. The existence of the communication agents makes the underlying network architecture transparent to system entities.

Database Management Agents

There is also a database management agent in each of the control computers which maintains and manipulates information for the system entities in the same control computer. Its duties include the update, retrieval, and manipulation of part and resource information, the creation, transaction recording, and destroying of part files, etc. When global knowledge about the manufacturing environment is needed, the database agent contacts the monitoring agent to obtain the most up-to-date global information. The global information will be buffered in the local database but has a lifetime that depends on the nature of the data. The consistency issue of the global information is solved by consulting monitoring agents and the communication agents periodically.

Communication

Although the model calls for heterarchical architecture, the underlying physical network does not need to be so. The basic network requirement for such a system is a communication network that supports reliable data transmission. The basic network functions are read, write, broadcast, and multicast. Any network protocol can be used as long as it guarantees the proper arrival and ordering of the messages at the application level.

The bidding communication protocols needed are bid construction and bookkeeping. The bid construction protocol includes bid announcement, acknowledgment, submission, and award. The bookkeeping protocol includes task arrival, file disposal, and information propagation.

The package types are task arrival, bid announcement, bid evaluation, bid submission, bid acknowledgment, bid award, bid rejection, file disposal, and information update.

For some manufacturing environments, high speed and high load capacity may be required for quick real-time control.

System Features

In this section, some important system features will be discussed.

Achieving Robustness

When an error such as machine failure, tool failure, fixture failure, or part failure occurs, the corresponding agent withdraws itself from the communication network or stops the bidding operation and may call for an error recovery module for diagnosis. The rest of the system can continue operation without disruption from the error.

To assure robustness when a computer goes down, each resource agent can keep a copy of the part file which it is currently serving. When the computer in which the part agent resides goes down, the current operation agent can detect the failure and act as the new part agent or help to create a new part agent for it. If the current operation resource computer is also down, the previous operation agents can be invoked. An age mechanism is used to maintain the files. The age is increased by one when a new operation is finished and moved to another machine. The part agent file is removed from the machine when it becomes too "old." The definition of the "old age to die" is system-dependent. For a reliable system, the old age can be set to a small number such as two or less. When the resource agent computer is down, a new agent can be created residing in another control computer based on the knowledge of the resource stored in other control computers. The resource and the related parts can either keep the commitment and continue the operation or restart the bidding process to look for a new part or resources.

Change Incorporation

When change occurs, only the corresponding agents are affected. The deletion of a physical component only causes the related agents to resubmit bids. For the ad-

Downloaded by [171.67.34.205] at 23:29 24 June 2013

dition of a new component, the only thing needed to be done is to attach a control module to it and connect it to the network. If the objective or the price of a part changes, or the data file of the resources needs to be changed, the corresponding agents can be found by tracing them through their paths or by broadcasting the change information with the corresponding part numbers or resource numbers to identify the affected entities. Algorithms can be added or deleted from the knowledge base easily by informing the intelligence agents. For most changes, continued operation of the system is guaranteed.

Deadlock Prevention

To avoid a part's being stuck in the system for a long time, an age factor can be added in the part objectives, and price functions and the corresponding weights will be increased when the part becomes old in the system. This will lead to the essential acceptance of the part to the resources.

Integration of Machine Tools

We separate the control flow, database, knowledge base, and network communication protocol in the proposed framework. In this way, the system can be implemented in an existing computer-aided manufacturing environment and be adapted easily to run in future factories. The integration of various machine tools from different vendors (which is an important requirement of a framework for scheduling and controlling the coordination of part flow and information flow [14]) can be done by the unified communication protocol.

Summary

We have described a generic framework for a control and scheduling system in a dynamic manufacturing environment based on task sharing and a negotiation metaphor by using the combination of objective and price mechanisms. Under this framework, the part agents make decisions based on the objective functions of the part, while the resource agents make decisions based on the price evaluation system. The goal of the part is to achieve the objectives with a minimum price paid, and the goal of the resource agent is to maximize the charged price to fulfill the part's requirements. The system functions based on the equilibrium of the objectives and prices under the generic bid construction model. A resource synchronization scheme is proposed to assure the efficient and simultaneous services of resources to a part. Mutual agreement is made through two-way communication between the parts and resources. Dynamic task decomposition is done automatically in the negotiation process.

The basic control modules are part agents, resource agents, intelligence agents, monitoring agents, communication agents, and database management agents. Part and resource agents are responsible for the control flow and decision making. They have the ability to make decisions autonomously. Intelligence agents are responsible for the manipulation of knowledge bases. The monitoring agent monitors the system and interfaces with other system components. The communication and database management agents do bookkeeping and manipulation of database, respectively. The control, decision making, knowledge base, and data base are all distributed throughout the system.

The framework is designed so that the structures of each control module are the same. Inevitable differences among modules are abstracted into the databases and knowledge bases. Modules communicate through a communication network with the same message passing protocol heterarchically. The amount of the software rewritten and the effect of system uncertainty are reduced to a minimum. The system is highly fault-tolerant. The separation of the part and machine intelligence from the bidding framework also makes it easy to incorporate new algorithms and to include any new parameters deemed important for decision making.

In summary, the proposed architecture is highly distributed, flexible, and effective and thus can be applied to the real time control and scheduling of open manufacturing systems.

Simulation Result Discussion

A generic object-oriented control and scheduling simulation system has been developed based on the proposed architecture. Figure 6 depicts the object hierarchy of the simulation system. The simulation system is coupled with a very high level programming language that contains constructs to model the manufacturing environment, flexible processing process plans, and the control structures and strategies. The modeling of the environment includes the definition of machines, transporters, tools, tool magazines, tool storage rooms, pallets/fixtures, pallet storage rooms, machine breakdowns, factory layout geometries, initial system states, and other factors common in manufacturing environments. By employing the language as the front end to the simulation system, very realistic modeling of manufacturing systems is possible, and the simulation system can be used to study the effects of different setups of manufacturing systems and various control strategies.

Several experiments using different price systems and

Downloaded by [171.67.34.205] at 23:29 24 June 2013

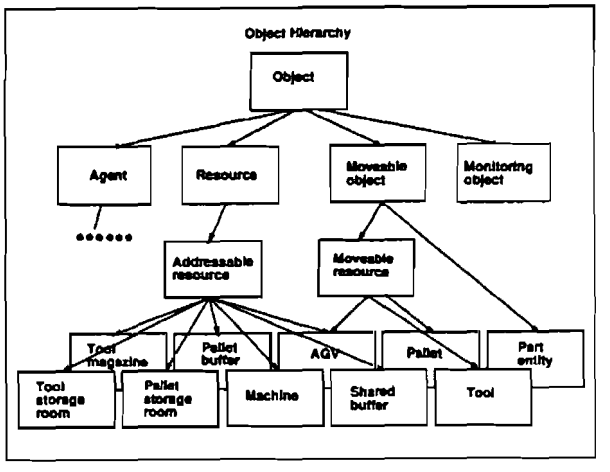


Figure 6. Object hierarchy

algorithms with various complexity have been tested. The simulation results show that the framework did provide a coordinated information and physical flow and can cope with changes such as resource breakdown, addition and removal of resources, and objective changes of parts in the examples tested. For example, Figure 7 shows how the price charged by each of five machines changes dynamically over time. In this figure, one of the machines, Mill 2, breaks down during the period from 20 to 30 hours. Notice how the prices adapt to this event. Figure 8 compares the average cost per part as it changes dynamically compared for three different situations: 1) under heavy traffic loads with machine failure (from hours 20 to 30), 2) under heavy traffic without failure, and 3) under lighter traffic loads with failure. Figure 9 shows how, under light traffic loads, the utilization of the five machines adjusts to the failure of Mill 2. We also compared the system performance

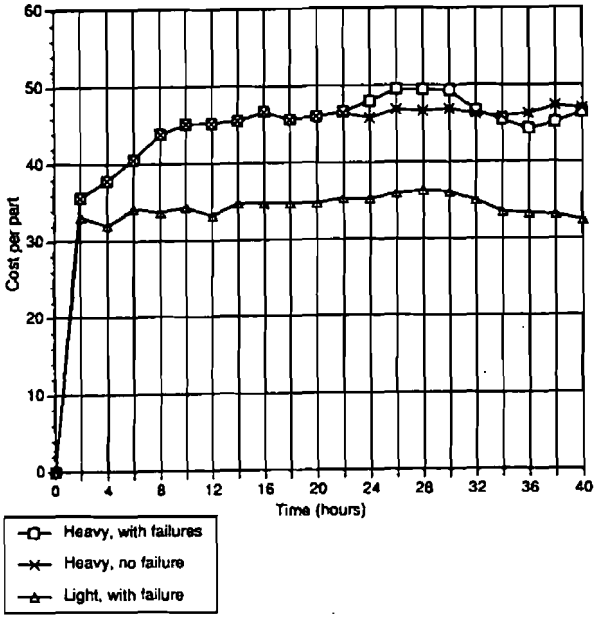


Figure 8. Part average cost

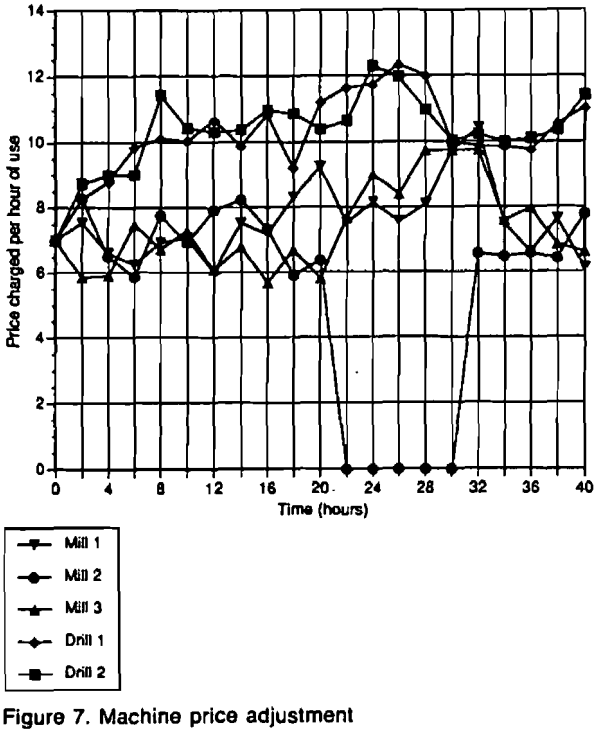


Figure 7. Machine price adjustment

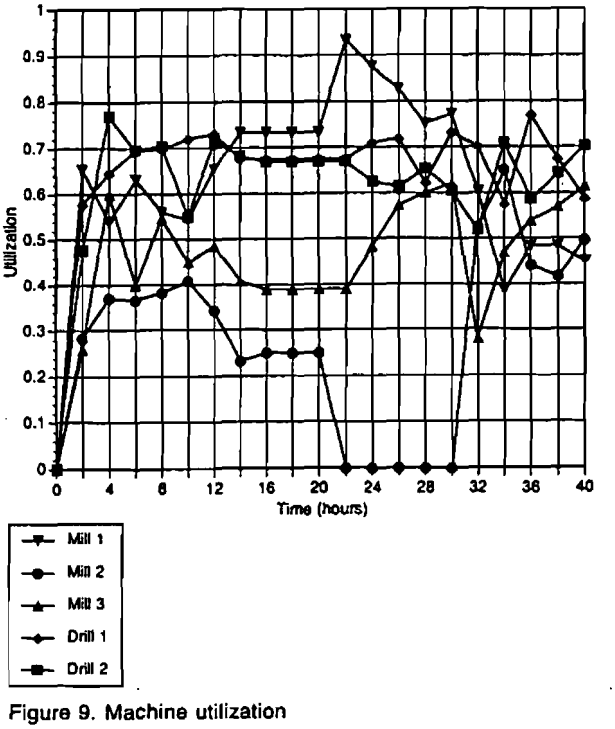


Figure 9. Machine utilization

Downloaded by [171.67.34.205] at 23:29 24 June 2013

to other distributed control systems such as Lewis's data flow control system and systems without considering secondary resources.

The overall results show that our system is superior to other systems tested in the sense that it achieves better individual goals and yields a better overall system performance. The price systems, algorithms, the implementation issues, and detailed results will be reported in other papers.

Conclusion

In this paper, we present a framework for integrating shop floor control using autonomous agents. The framework utilizes distributed decision making and information flow and highlights competition and cooperation among heterogeneous controls. An efficient resource unification scheme is developed to unify and coordinate the usage of the resources in which critical resources play a central role in decision making. A generic bid construction based on objectives and a price mechanism is proposed. Mutual selection and mutual agreement among parts and resources are made through multiple step and multiple way negotiation. An object oriented architecture was developed based on the proposed modeling. Flow orchestration, system equilibrium, communication requirements, and change incorporation issues are also addressed.

In summary, the proposed framework allows jobs and resources to have their own intelligent controls to match individual needs. Control strategies can be modeled and modified and new control strategies can be incorporated into the system easily. The flexibility of the system allows the system to be adapted to different manufacturing environments and to respond to changing objectives well. An object-oriented simulation system is built to demonstrate the flexibility and effectiveness of the proposed framework. The results show that the proposed framework provides a foundation for highly adaptive, real time shop floor control.

Acknowledgments

This work was supported by the Engineering Research Center for Intelligent Manufacturing Systems under a grant from the National Science Foundation, number CDR 8803017, with additional funding from the Defense Advanced Research Projects Agency (DARPA).

REFERENCES

[1] Abdus, J. B., Barbara, A. J., and Nagel, R. N.," Theory and Practice of Hierarchical Control," *23rd IEEE Computer Society International Conference*, 18-39 (September 1981).

[2] Adams, Joseph, Balas, Egon, and Zawack, Daniel, "The Shifting Bottleneck Procedure for Job Shop Scheduling," *Management Science*, 34, 3 (March 1988).

[3] Browne, Jim, Dubois, Didier, Rathmill, Keith, Sethi, Suresh, and Stecke, Kathryn E., "Classification of Manufacturing Systems," *The FMS Magazine*, 114-117 (April 1984).

[4] Cox, Brad J., *Object Oriented Programming: An Evolutionary Approach*, Addison-Wesley Publishing Company, Mass. (1986).

[5] Davis, Randall, and Smith, Reid G., "Negotiation as a Metaphor for Distributed Problem Solving," *Artificial Intelligence*, 20, 63-109 (1983).

[6] Duffie, Neil A., and Piper, Rex S., "Nonhierarchical Control of Manufacturing Systems," *Journal of Manufacturing Systems*, 5, 2, 137-139 (1986).

[7] Farber, D. J., and Larson, K. C., "The Structure of a Distributed Processing System—Software," *Proceedings of the Symposium on Computer Communication Networks and Teletraffic*, Polytechnic Institute of Brooklyn (April 1972).

[8] Farber, D. J., *et al.*, "The Distributed Computer System," *Proceedings of the Seventh Annual IEEE Computer Society International Conference*, (February 1973).

[9] Ferguson, Donald, Yemini, Yechiam, and Nikolaou, Christos, "Microeconomic Algorithms for Load Balancing in Distributed Computer Systems," *International Conference on Distributed Computer Systems, IEEE*, 491-499 (1988).

[10] Fox, Bob, "OPT: An Answer for America, Part I, II," *Inventories and Production*, 2, 4/6 (1982).

[11] Fox, Bob, "OPT: An Answer for America, Part III," *Inventories and Production*, 3, 1 (1983).

[12] Fox, B. R., and Kempf, K. G., "Complexity, Uncertainty and Opportunistic Scheduling," *Proceedings of the IEEE Second Conference on Artificial Intelligence Applications: The Engineering of Knowledge-Based Systems*, Miami Beach, Fl, 487-492 (December 1985).

[13] Harvany J., "Intelligence and Cooperation in Heterarchical Manufacturing Systems," *Proceedings of the 16th CIRP International Seminar on Manufacturing Systems*, Tokyo, Japan, 1-4 (1984)

[14] Keller, Erik L., "Air Force in CIM Framework Drive," *Manufacturing Week*, p. 11 (March 23, 1987).

[15] Lewis, William C., Jr., "Data Flow Architectures for Distributed Control of Computer Operated Manufacturing Systems: Structure and Simulated Applications," Ph.D. Dissertation, School of Industrial Engineering, Purdue University, West Lafayette, IN (May 1981).

[16] Lewis, William C., Jr., Barash, M. M., and Solberg, J. J., "Computer Integrated Manufacturing System Control: A Dataflow Approach," *Journal of Manufacturing Systems*, 2 (1987).

[17] Lin, Grace Yuh-Jiun, and Solberg, James J., "Flexible Routing Control and Scheduling," *Proceedings of the Third ORSA/TIMS Conference on Flexible Manufacturing Systems* (K.E. Stecke and R. Suri, editors) Elsevier, Amsterdam, 155-160 (1989).

[18] Lin, Grace Yuh-Jiun, and Solberg, James J., "Effectiveness of Flexible Routing Control," *International Journal of Flexible Manufacturing Systems*, 3, 3/4, 189-212 (June 1991).

[19] Lin, Grace Y., "An Opportunistic Price-Based Multiple-Resource and Part Scheduling," Ph.D. Dissertation, School of Industrial

Downloaded by [171.67.34.205] at 23:29 24 June 2013

Engineering, Purdue University, West Lafayette, IN (August 1992).

[20] Maley, James G., "Part Flow Orchestration in Distributed Manufacturing Processing ," Ph.D. Dissertation, School of Industrial Engineering, Purdue University, West Lafayette, IN (August 1987).

[21] Maley, James G., and Solberg, James J., "Part Flow Orchestration in CIM," *Proceedings of the 9th International Conference on Production Research*, Cincinnati, OH, 17-20 (August, 1987).

[22] Malone, T. W., Fikes, R. E., and Howard, M. T., "Enterprise: A Market-Like Task Scheduler for Distributed Computing Environments," CISR WP #111, Sloan WP #1537-84, Center for Information Systems Research, Sloan School of Management, MIT (October 1983).

[23] Mclean, Charles, Mitchell, Mary, and Barkmeyer, Edward, "A Computer Architecture for Small-Batch Manufacturing," *IEEE Spectrum*, 59-64 (May 1983).

[24] Morton, T. E., Lawrence, S. R., Rajagopolan, S., and Kekre, S., "A Price-Based Shop Scheduling Module," *Journal of Manufacturing and Operations Management*, 1, 2 (Summer 1988).

[25] Nygaard, Kristen, "Basic Concepts in Object Oriented Programming," *SIGPLAN Notices*, 21, 10, 128-132 (October 1986).

[26] Ow, Peng Si, Smith, Stephen F., and Howie, Raddy, "A Cooperative Scheduling System," *Proceedings of the Second International Conference on Expert Systems and the Leading Edge in Production Planning and Control*, 43-56 (May 1988).

[27] Ow, Peng Si, and Smith, Stephen F, "Viewing Scheduling as an Opportunistic Problem-Solving Process," *Annals of Operations Research*, 12, 85-108, (1988).

[28] Parunak, H. V. D., White, J. F., Lozo, P., Judd, R., Irish, B., and Kindrick, J., "An Architecture for Heuristic Factory Control," *Proceedings of the American Control Conference*, Seattle (1986).

[29] Parunak, H. V. D., "Manufacturing Experience with the Contract Net," *Distributed Artificial Intelligence*, Michael M. Huhns, Pitman, London, 285-310 (1987).

[30] Ramamritham, K., and Stankovic, John A., "Dynamic Task Scheduling in Hard Real-Time Distributed Systems," *IEEE Software*, 65-75 (July 1984).

[31] Rentsch, Tim, "Object Oriented Programming," *SIGPLAN Notices*, 17, 9, 51-57 (September 1982).

[32] Sadeh, Norman, and Fox, M. S., "CORTES: An Exploration into Micro-Opportunistic Job-Shop Scheduling," *Workshop on Manufacturing Production Scheduling*, IJCAI89, Detroit (1989).

[33] Shaw, Michael J. P., "The Design of a Distributed Knowledge-Based System for the Intelligent Manufacturing Information System," Ph.D Dissertation, Purdue University, West Lafayette, Indiana (August, 1984).

[34] Shaw, Michael J. P., and Whinston, Andrew B., "A Distributed Knowledge-based Approach to Flexible Automation: The Contract Net Framework," *International Journal of Flexible Manufacturing Systems*, 1, 1 (1989).

[35] Smith, Reid G., "The Contract Net Protocol: High-Level Communication and Control in a Distributed Problem Solver," *IEEE Transactions on Computers*, C-29, 12, 1104-1113 (December 1980).

[36] Smith, Reid G., and Davis, Randall, "Frameworks for Cooperation in Distributed Problem Solving," *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-11, 1, 61-70 (January 1981).

[37] Stankovic, John A., and Sidhu, Inderjit S., "An Adaptive Bidding Algorithm for Process Clusters and Distributed Groups," *Proceedings of the 4th International Conference on Distributed Computing Systems, IEEE Computer Society*, 49-59 (1984).

[38] Solberg, James J., "Production Planning and Scheduling in CIM," *Proceedings of the 11th World Computer Congress, IFIP*, San Francisco, CA (August 1989).

[39] Ting, James J., "A Cooperative Shop Floor Control Model for Computer-Integrated Manufacturing," *Proceedings of CIMCON'90*, Albert Jones, Editor, U.S. Government Printing Office, Washington,D.C., 446-465 (1990).

[40] Upton, David M., "The Operation of Large Computer Controlled Manufacturing Systems," Ph.D. Dissertation, School of Industrial Engineering, Purdue University, West Lafayette, IN (December 1989).

[41] Vamos, T., "Co-operative Systems—An Evolutionary Perspective," *IEEE Control Systems Magazine*, 3, 2, 9-14 (1983).

[42] Waldspurger, Carl A., Hoff, Tad, Huberman, Bernardo A., Kephart, Jeffrey O., and Stornetta, Scott, "SPAWN: A Distributed Computational Economy," Dynamics of Computation Group, Xerox Palo Alto Research Center, Palo Alto, CA (1989).

Grace Yuh-jiun Lin is a Ph.D. candidate in the School of Industrial Engineering. Her research interests include manufacturing control and scheduling, computer integrated manufacturing, distributed decision making, and artificial intelligence. She has taught several undergraduate and graduate courses at Purdue. She is a member of IIE, ORSA, TIMS, and the honor societies of Phi Kappa Phi and Omega Rho.

James J. Solberg is the Director of the Engineering Center for Intelligent Manufacturing Systems and a Professor of Industrial Engineering at Purdue University in West Lafayette, Indiana. He received a B.A. degree from Harvard in mathematics in 1964, an M.A. in mathematics and an M.S. in industrial engineering from the University of Michigan in 1967, and a Ph.D. in industrial engineering from the University of Michigan in 1969.

In addition to several teaching awards, Dr. Solberg won the "Book of the Year" Award in 1977 and the David F. Baker Distinguished Research Award from the Institute of Industrial Engineers in 1982. He is a member of the National Academy of Engineering.

As Director of the Engineering Research Center for Intelligent Manufacturing Systems since its formation in 1985, Dr. Solberg has managed a cross-disciplinary program in research, education, and technology transfer involving some forty professors and 200 students annually.

Downloaded by [171.67.34.205] at 23:29 24 June 2013